# BIZEV-02: Instrumentation

> **Series:** BIZEV — Business Events & Funnel Analysis | **Notebook:** 2 of 7 | **Created:** March 2026 | **Last Updated:** 08/04/2026

## Overview

Capturing meaningful business events requires deliberate instrumentation. This notebook covers the practical techniques for generating business events — from OneAgent auto-detection of web request data, to custom API ingestion, SDK integration, and OpenTelemetry span-to-bizevent mapping. You will also learn best practices for event naming conventions, payload design, and cardinality management to keep your business analytics clean and performant.

---

## Table of Contents


1. OneAgent Auto-Detection
2. Business Events API Ingestion
3. SDK Integration
4. OpenTelemetry Span-to-Bizevent Mapping
5. Event Naming Best Practices
6. Payload Design and Cardinality
7. Verifying Instrumentation with DQL
8. Summary and Next Steps


---

# Prerequisites

| Requirement | Details |
|---|---|
| **Dynatrace Environment** | SaaS with Grail enabled |
| **Permissions** | `storage:bizevents:read`, `bizevents.ingest` |
| **OneAgent** | Deployed on at least one application host (for auto-detection) |
| **Knowledge** | BIZEV-01 fundamentals, basic REST API concepts |

> **No Grail yet?** Every capture path in this notebook writes to the Grail `bizevents` table and therefore requires a SaaS tenant with Grail. For what a classic (Gen2) environment can do instead — request attributes, session/action properties + USQL, log metrics — and for the hybrid pattern that adopts these capture paths with only a minimal Gen3 footprint, see **BIZEV-07: Gen2 vs Gen3 — Business Events Without (or Before) the Full Move**.

# 1. OneAgent Auto-Detection

OneAgent can automatically capture business events from web application requests using **capture rules**. This is the lowest-effort approach — no code changes required.

## Setting Up Capture Rules

Navigate to **Settings > Business Analytics > OneAgent > Capture rules** and define:

| Setting | Description | Example |
|---|---|---|
| **Trigger** | URL pattern or request attribute match | `/api/v1/orders/*` |
| **Event type** | The `event.type` value to assign | `com.myapp.order.created` |
| **Data extraction** | Request/response attributes to capture | `order_id`, `total_amount` |
| **Filtering** | Conditions to limit capture | HTTP method == `POST` |

### What Gets Captured Automatically

When a capture rule matches, OneAgent creates a business event with:

- The configured `event.type`
- `event.provider` set to the application name
- Extracted request attributes as custom payload fields
- Automatic correlation with the trace context (span ID, trace ID)

> **Tip:** Start with auto-detection for existing web applications, then add custom instrumentation for backend processes that don't have HTTP endpoints.

```
// Check which business events are being captured by OneAgent auto-detection
// Events from OneAgent typically have the application name as event.provider
fetch bizevents, from:-24h
| filter isNotNull(dt.entity.service)
| summarize event_count = count(), by:{event.type, event.provider}
| sort event_count desc
| limit 15
```

## 2. Business Events API Ingestion

The Business Events API allows you to send events from any system — backend services, batch processes, third-party integrations, or IoT devices.

### CloudEvents Format

The API expects events in CloudEvents format:

```
{
  "specversion": "1.0",
  "id": "evt-20260312-001",
  "type": "com.myapp.payment.processed",
  "source": "payment-service",
  "time": "2026-03-12T10:30:00Z",
  "data": {
    "payment_id": "PAY-98765",
    "amount": 249.99,
    "currency": "USD",
    "method": "credit_card",
    "customer_tier": "gold"
  }
}
```

## API Endpoint

```
POST https://{environment-id}.live.dynatrace.com/api/v2/bizevents/ingest
Content-Type: application/cloudevents+json
Authorization: Api-Token dt0c01.xxxxx
```

## Batch Ingestion

For high-volume scenarios, send multiple events in a single request using `application/cloudevents-batch+json`:

```
[
  {"specversion": "1.0", "type": "com.myapp.event1", "source": "svc", "data":
{}},
  {"specversion": "1.0", "type": "com.myapp.event2", "source": "svc", "data":
{}}
]
```

> **Important:** The API has rate limits. For sustained high throughput (>1000 events/second), use batch ingestion and implement retry logic with exponential backoff.

```
// Verify API-ingested events are arriving
// API-ingested events often have a custom source/provider value
fetch bizevents, from:-1h
| summarize {event_count = count(),
           latest = max(timestamp),
           earliest = min(timestamp)}, by:{event.provider}
| sort event_count desc
```

## 3. SDK Integration

The Dynatrace OneAgent SDK provides language-specific methods to send business events directly from application code.

### Java Example

```
import com.dynatrace.oneagent.sdk.api.BusinessEventsOneAgentApi;

Map&lt;String, Object&gt; attributes = new HashMap&lt;&gt;();
attributes.put("order_id", "ORD-12345");
attributes.put("total", 149.99);
attributes.put("items", 3);

BusinessEventsOneAgentApi.sendBizEvent(
    "com.myapp.checkout.completed",
    attributes
);
```

### JavaScript / Node.js Example

```
const dtrum = window.dtrum;

dtrum.sendBizEvent('com.myapp.add-to-cart', {
  product_id: 'PROD-456',
  product_name: 'Widget Pro',
  price: 29.99,
  quantity: 2
});
```

### Python Example

```
import oneagent

sdk = oneagent.get_sdk()
sdk.send_biz_event(
    event_type="com.myapp.signup.completed",
    attributes={
        "user_id": "USR-789",
        "plan": "premium",
        "source_campaign": "spring-2026"
    }
)
```

> **Tip:** SDK-generated events are automatically correlated with the active PurePath, linking business events to the full distributed trace.

## 4. OpenTelemetry Span-to-Bizevent Mapping

If your application already produces OpenTelemetry spans with business-relevant attributes, you can map them to business events using **OpenPipeline processing rules**.

### How It Works

1. Application emits OTel spans with business attributes (e.g., `order.id`, `order.total`)
2. OpenPipeline receives the span data
3. A processing rule extracts business attributes and routes them to `bizevents`
4. The original span remains in `spans` — the business event is an additional record

### OpenPipeline Configuration

```
# Example OpenPipeline rule for span-to-bizevent mapping
pipelines:
  - name: "Extract checkout events from spans"
    processing:
      - type: bizevent
        condition: "span.name == 'checkout.complete'"
        eventType: "com.myapp.checkout.completed"
        attributes:
          - source: "order.id"
            target: "order_id"
          - source: "order.total"
            target: "amount"
```

> **Note:** This approach avoids double-instrumentation. If your spans already carry business data, use OpenPipeline rather than adding SDK calls.

```
// Look for business events that may have originated from spans
// These often have trace/span correlation fields
fetch bizevents, from:-24h
| filter isNotNull(trace_id) or isNotNull(span_id)
| summarize correlated_count = count(), by:{event.type}
| sort correlated_count desc
| limit 10
```

## 5. Event Naming Best Practices

A consistent naming convention is critical for long-term analytics. Poor naming leads to fragmented data and unreliable queries.

### Recommended Naming Convention

Use reverse-domain notation with a verb suffix:

```
com.&lt;company&gt;.&lt;domain&gt;.&lt;action&gt;
```

| Pattern | Example | Description |
|---|---|---|
| `com.myapp.order.created` | Order placed | New order in system |
| `com.myapp.order.completed` | Order fulfilled | Order shipped/delivered |
| `com.myapp.payment.processed` | Payment received | Successful payment |
| `com.myapp.payment.failed` | Payment rejected | Failed payment attempt |
| `com.myapp.user.signup` | New registration | User account created |
| `com.myapp.user.login` | Authentication | Successful login |
| `com.myapp.cart.updated` | Cart change | Item added/removed |

### Naming Anti-Patterns

| Avoid | Why | Better |
|---|---|---|
| `order` | Too vague — created? cancelled? | `com.myapp.order.created` |
| `OrderCreated` | Inconsistent casing | `com.myapp.order.created` |
| `com.myapp.order.created.v2` | Version in name fragments data | Use payload versioning |
| `event_12345` | Meaningless identifier | Use descriptive domain names |

```
// Audit event naming — find event types that may need standardization
fetch bizevents, from:-7d
| summarize {event_count = count(),
           first_seen = min(timestamp),
           last_seen = max(timestamp)}, by:{event.type}
| sort event_count desc
```

## 6. Payload Design and Cardinality

The custom attributes you include in business events determine what analytics are possible. Design payloads carefully.

## Payload Design Principles

| Principle | Guidance |
|---|---|
| **Include business identifiers** | `order_id`, `customer_id`, `session_id` |
| **Include measurable values** | `amount`, `quantity`, `duration_ms` |
| **Include categorical dimensions** | `category`, `region`, `customer_tier` |
| **Avoid high-cardinality strings** | Don't include full URLs, stack traces, or free text |
| **Use consistent types** | Always send `amount` as a number, not sometimes as a string |

## Cardinality Management

High-cardinality fields (fields with millions of unique values) can degrade query performance.

| Field Type | Cardinality | Impact |
|---|---|---|
| `customer_tier` (gold/silver/bronze) | Low (~3 values) | Excellent for `summarize by:` |
| `product_category` | Medium (~100 values) | Good for grouping |
| `order_id` | High (millions) | Fine for lookup, avoid `summarize by:` |
| `full_url` | Very high | Avoid — parse into structured fields instead |

> **Warning:** Using `summarize by:{order_id}` on millions of unique order IDs will produce a massive result set and slow queries. Use high-cardinality fields for filtering (`filter order_id == "ORD-123"`) rather than grouping.

```
// Assess cardinality of common fields in your business events
fetch bizevents, from:-24h
| summarize {total = count(),
           distinct_types = countDistinct(event.type),
           distinct_providers = countDistinct(event.provider),
           distinct_categories = countDistinct(event.category)}
```

## 7. Verifying Instrumentation with DQL

After setting up instrumentation, validate that events are arriving correctly and contain the expected data.

```
// Check for gaps in business event ingestion over the last 24 hours
// A healthy pipeline should show consistent event flow
fetch bizevents, from:-24h
| makeTimeseries event_count = count(), interval:15m
```

```
// Validate that required fields are populated (not null)
fetch bizevents, from:-1h
| summarize {total = count(),
           has_type = countIf(isNotNull(event.type)),
           has_provider = countIf(isNotNull(event.provider)),
           has_category = countIf(isNotNull(event.category))}
| fieldsAdd type_pct = round(toDouble(has_type) / toDouble(total) * 100,
decimals: 1),
           provider_pct = round(toDouble(has_provider) / toDouble(total) *
100, decimals: 1),
           category_pct = round(toDouble(has_category) / toDouble(total) *
100, decimals: 1)
```

## 8. Summary and Next Steps

In this notebook, you learned:

- **OneAgent auto-detection** — Capture business events from web requests without code changes
- **API ingestion** — Send events from any system using CloudEvents format
- **SDK integration** — Emit events directly from application code with trace correlation
- **OpenTelemetry mapping** — Route span data to bizevents via OpenPipeline

- **Naming conventions** — Use reverse-domain notation with action verbs
- **Payload design** — Balance richness with cardinality management

## Next Steps

- **BIZEV-03: Funnel Analysis** — Use your instrumented events to build conversion funnels
- **BIZEV-05: KPIs and Metrics** — Extract business KPIs from event data

## References


- Environment API (DT docs)
- OneAgent SDK for Business Events
- OpenPipeline Processing


---